Documentação do Trabalho Prático de Automação em Tempo Real - Etapa I

Marcele Araponga, Vitória Helena Frederico
Departamento de Engenharia Eletrônica
Universidade Federal de Minas Gerais
Belo Horizonte, Brasil
louise20marcele03@gmail.com, vitoriahelena.311005@gmail.com

Outubro de 2026

Sumário

1 Introdução **1**

2 Objetivos da Etapa I **2**

3 Decisões de Projeto e Arquitetura **2**

4 Sincronização e Controle de Fluxo por Eventos **5**

5 Implementação da Lista Circular 1 e Controle de Concorrência **8**

1 Introdução

A presente documentação registra a primeira etapa (Etapa I) do desenvolvimento de um sistema *multithreading*, no qual se propõe a integrar, eficientemente, os sistemas de **Planejamento e Controle de Produção** (PCP) e **Supervisório** (SCADA), para garantir o acompanhamento em tempo real de uma linha de produção.

O código foi desenvolvido em linguagem *C++* na plataforma *Windows* (API WIN32), utilizando o ambiente de desenvolvimento *Visual Studio*.

2 Objetivos da Etapa I

- **Modelagem da arquitetura e disparo da aplicação:** distribuição conceitual das tarefas em processos ou *threads*, inicializadas a partir de um processo principal, e distribuição física, em termos de código, em projetos de uma solução;
- **Controle de fluxo via sinalização de eventos:** implementação de mecanismos de bloqueio e desbloqueio individual de tarefas, incitados pela intervenção externa do usuário;
- **Implementação da 1ª Lista Circular em Memória:** fila circular em memória RAM com capacidade para 100 (cem) mensagens;
- **Temporização provisória:** uso da função *Sleep()* para simulação dos comportamentos das tarefas de leitura, a ser substituída na próxima etapa.

3 Decisões de Projeto e Arquitetura

A tabela 1 reúne as 6 (seis) tarefas a serem realizadas pela aplicação que, presentemente, propõe-se implementar.

Tarefa	**Responsabilidade**	**Uso de buffers**
1. Leitura do supervisório	Produzir mensagens de processo a cada 500 ms na etapa final	Buffer 1
2. Leitura do PCP	Produzir mensagens de programação em intervalos aleatórios de 1 a 5 segundos	Buffer 1
3. Captura e distribuição	Retirar mensagens do primeiro buffer e encaminhá-las ao segundo buffer ou à tarefa de gestão da produção	Buffers 1 e 2
4. Exibição do processo	Consumir mensagens do segundo buffer e apresentá-las em um console dedicado	Buffer 2
5. Gestão da produção	Consumir mensagens do PCP e exibi-las em outro console dedicado	Mensagens do PCP
6. Interação do usuário com o teclado	Tratar comandos de pausa, retomada, bloqueio, desbloqueio e encerramento	Buffer do dispositivo de entrada padrão

Tabela 1: Tabela de tarefas e suas funções, além de a que estruturas de dados cada uma faz acesso.

Primeiramente, faz-se necessária a "[...] definição de como as tarefas anteriormente descritas serão modeladas em termos de processos e threads" (1).

Trata-se de um requisito de projeto designar um processo principal, a partir do qual os demais são disparados, por meio da função *CreateProcess()*.

Portanto, implementou-se a função *inicializarProcesso(processo &processo)*, que recebe uma referência a uma instância da *struct* que reúne as informações de um processo, inicializa todos os processos na estrutura e os cria.

```
bool inicializarProcesso(processo &processo) {

    processo.processStatus = FALSE;
    ZeroMemory(&processo.si, sizeof(processo.si));
    ZeroMemory(&processo.newProcessInfo, sizeof(processo.
    newProcessInfo));
    processo.si.cb = sizeof(processo.si);

    processo.processStatus = CreateProcess(
        processo.processPath.c_str(),
        NULL,
        NULL,
        NULL,
        FALSE,
        NORMAL_PRIORITY_CLASS,
        NULL,
        NULL,
        &processo.si,
        &processo.newProcessInfo);
    }
```

Listing 1: Função *inicializarProcesso()*.

Na função *main()*, a estrutura com os processos é iterada em *loop*: se a inicialização de pelo menos um dos processos falhar, inicia-se o procedimento de encerramento dos processos-filhos, encapsulado na função de nome *encerrarProcessosFilhos()*, e aborta-se a execução atual.

É conveniente que esse mesmo processo seja, também, o responsável por coordenar a mudança de fluxo de execução, mediante a sinalização de eventos (o que será melhor abordado na seção seguinte).

Dessa forma, o processo principal ficou incumbido de:

1. disparar os demais processos,
2. interceptar os comandos fornecidos pelo usuário através do dispositivo padrão de entrada (teclado) e
3. coordenar o fluxo de execução do programa, através de objetos eventos.

Percebe-se que o processo precisa, concomitantemente, estar aberto para receber comandos do usuário e, também, monitorar a integridade dos processos filhos, isto é, rastrear a possibilidade de algum deles terminar prematuramente em função de quaisquer erros em sua execução.

Assim sendo, criou-se uma *thread* secundária do processo principal, responsável por executar a função *monitorarProcessosFilhos()*. A função *menu()*, por sua vez, espera por uma entrada do usuário. Enquanto isso não acontece, a cada 100 (cem) milisegundos, ela confere se o evento de encerramento prematuro de um processo-filho fora sinalizado. Se sim, ela inicia a rotina de encerramento dos demais processos-filhos.

No que tange às tarefas 4 e 5, como deseja-se dois consoles independentes, um para cada tarefa, optou-se pela organização em dois processos separados. Dessa forma, a tarefa 4 será realizada pelo processo de nome "**processoDeExibicao**", que consta no projeto de mesmo nome na solução; a tarefa 5 será realizada pelo processo encontrado no projeto homônimo, "**processoGestaoDeProducao**".

Ambas as tarefas 3 e 4 acessam o buffer 2, cuja implementação, como consta no enunciado, há de ser em uma região de memória compartilhada entre processos. Esse requisito de projeto facilitou uma última decisão a ser tomada, a de em qual processo a tarefa 3 seria alocada; o fator de maior peso na decisão supracitada seria a conveniência de as tarefas 3 e 4 compartilharem um processo. Como essa não é uma opção, aglutinar as tarefas 1, 2 e 3 em único processo, já que todas acessam o buffer 1, serve ao propósito da simplicidade de implementação.

A tabela 2 especifica de qual domínio conceitual cada componente participa na arquitetura proposta.

Componente	**Domínio**
Tarefa 1 (Supervisório)	Thread do processo produtor
Tarefa 2 (PCP)	Thread do processo produtor
Tarefa 3 (Captura e distribuição)	Thread do processo produtor
Buffer 1	Memória do processo produtor
Buffer 2	Memória compartilhada entre os processos produtor e de exibição
Tarefa 4 (Exibição)	Processo de exibição
Tarefa 5 (Gestão de produção)	Processo de gestão de produção
Tarefa 6 (Interação com o usuário)	Processo principal

Tabela 2: Tabela com a designação de cada componente na proposta de arquitetura.

O diagrama 1 permite a visualização de como os processos se relacionam entre si e como se relacionam com as estruturas de dados em memória.

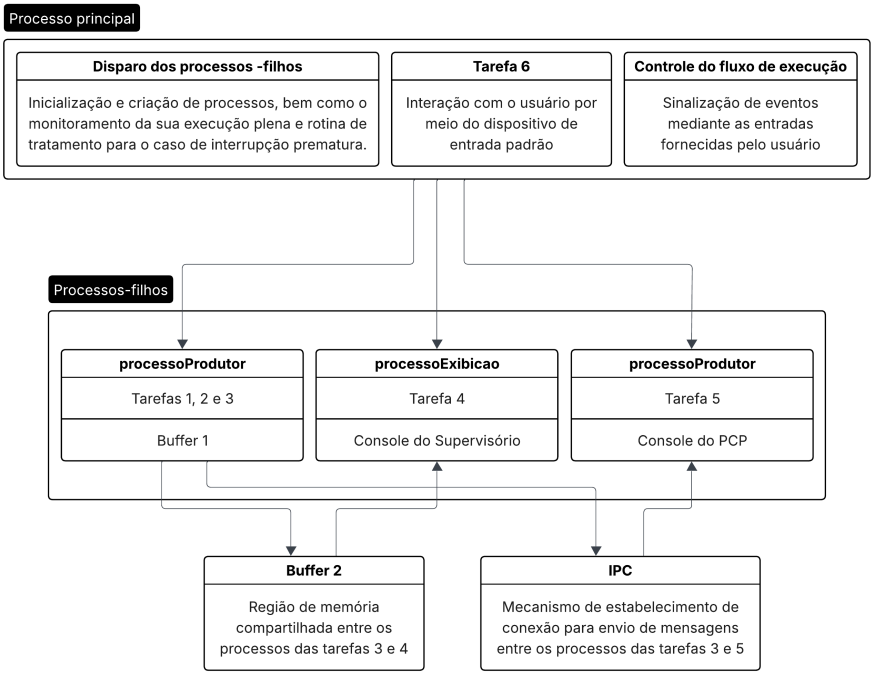


Figura 1: Diagrama da proposta de arquitetura.

4 Sincronização e Controle de Fluxo por Eventos

O processo principal, além de disparar os demais processos, por meio de um sistema de menu de opções e de captura de entradas fornecidas pelo usuário, permite o controle do bloqueio e desbloqueio das tarefas. A tabela 3 reúne os comportamentos que deseja-se que a aplicação seja capaz de realizar e o respectivo caractere que os engatilham.

Caractere	Comportamento
s	Alterna entre bloquear e retomar a leitura do Supervisório
m	Alterna entre bloquear e retomar a leitura do sistema de PCP
r	Alterna entre bloquear e retomar a retirada de mensagens
p	Alterna entre bloquear e retomar a exibição de dados de processo
g	Alterna entre bloquear e retomar a exibição da gestão da produção
1	Solicita a limpeza da janela da tarefa de exibição de dados de processo
2	Solicita a limpeza da janela da tarefa de gestão da produção
ESC	Solicita o encerramento de todas as tarefas e da leitura do teclado

Tabela 3: Tabela com a ação a ser realizada a depender da entrada fornecida pelo usuário.

Na função *menu()*, um bloco de comandos condicionais intercepta o comando do usuário. A depender de qual bloco condicional o programa adentra, um objeto do tipo evento é sinalizado pelo processo principal.

```
eventos.eventoDeEncerramentoDosProcessosFilhos = CreateEventW(
                nullptr,
                true,
                false,
                L"eventoDeEncerramentoDosProcessosFilhos");

handlesDosEventos.push_back(
                eventos.eventoDeEncerramentoDosProcessosFilhos);
```

Listing 2: Criação do evento de encerramento dos processos-filhos no processo principal.

```
if (tecla == ESC) {
    std::cout << "O sistema estah sendo encerrado." << std::endl;
    encerrarProcessosFilhos(processos);
}
```

Listing 3: Sinalização do evento de encerramento dos processos-filhos na função *menu()*.

```
void encerrarProcessosFilhos(std::vector<std::pair<processo*, std::
    string>> &processos) {

  if (eventos.eventoDeEncerramentoDosProcessosFilhos != nullptr) {
```

```
    if (!SetEvent(eventos.eventoDeEncerramentoDosProcessosFilhos))
    {
      std::cout << "Erro ao sinalizar o evento de encerramento dos
    processos-filhos. Codigo de saida: " << GetLastError() << std::
    endl;
    };

        [...]
```

Listing 4: Primeiras linhas de comando da função *encerrarProcessosFilhos()*.

Nos processos-filhos, por sua vez, realiza-se a abertura do evento, por meio do nome deste, o que torna o processo-filho sensível À sinalização do dito evento pelo processo principal.

```
HANDLE eventoDeEncerramento = OpenEventW(
    SYNCHRONIZE,
     FALSE,
     L"eventoDeEncerramentoDosProcessosFilhos");
```

Listing 5: Abertura do evento de encerramento em um processo-filho.

Finalmente, nos processos-filhos, uma thread dedica-se a executar a função *monitorarEncerramento()* que, por meio de *WaitForSingleObject()*, espera pela sinalização do evento de encerramento. Uma vez que isso aconteça, os processos-filhos iniciam suas rotinas de encerramento.

```
unsigned __stdcall monitorarEncerramento(void* param) {

    HANDLE eventoDeEncerramento = static_cast<HANDLE>(param);

      if (eventoDeEncerramento == nullptr) {
                  std::cout <<
                    "Hanlde do evento invalido. Codigo de saida: "
                    << GetLastError()
                    << std::endl;
        return 1;

      } else {

        DWORD result = WaitForSingleObject(
                            eventoDeEncerramento, INFINITE);

            [...]
```

Listing 6: Função *monitorarEncerramento()* dos processos-filhos

5 Implementação da Lista Circular 1 e Controle de Concorrência

O buffer 1 se trata de uma "[...] lista circular em memória, com o depósito de mensagens na mesma pelas tarefas de leitura do Supervisório e do sistema de PCP, e a retirada das mesmas pela tarefa de retirada de mensagens, levando em conta aspectos de sincronização eventualmente necessários"([1]).

Por questões de simplicidade de implementação, mostrou-se conveniente reunir as **tarefas 1**, **2** e **3** no mesmo processo que o buffer 1, uma vez que todas realizam acesso ao mesmo.

A implementação do buffer, em termos de código, utiliza de ponteiros de inserção (*end_*) e remoção (*begin_*) que, ao atingir o limite da capacidade (100 [cem] mensagens), retornam ao início da estrutura, emulando a impressão de circularidade.

Para garantir a que a condição de exclusão mútua seja satisfeita, a implementação faz uso de diretivas de sincronização, como *mutexes* e *semáforos*: aqueles intermediam o acesso aos recursos compartilhados, estabelecendo uma seção crítica e controlando o acesso das threads a ela, enquanto esses controlam o espaço disponível para que as **tarefas 1** e **2** escrevam na lista, e sinalizam para a disponibilidade de novos itens a serem consumidos pela **tarefa 3**.

Por fim, utilizou-se a função *Sleep(1000)* nas **tarefas 1** e **2** para simular a chegada de dados em tempo real. Com ela, o comportamento é periódico mas, na Etapa II de desenvolvimento, a função será substituída afim de imprimir o caráter inconstante da chegada de dados, característico de aplicações de tempo real.

Referências

[1] L. T. S. Mendes, "Trabalho final – 2026/2: Automação em tempo real." Escola de Engenharia, Departamento de Engenharia Eletrônica, Universidade Federal de Minas Gerais, 2026. Arquivo PDF disponível em ambiente virtual de aprendizagem.